

Advanced Topics in Computer Graphics I

Photon Mapping

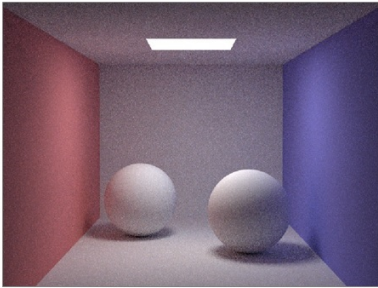


Prof. Dr. Reinhard Klein

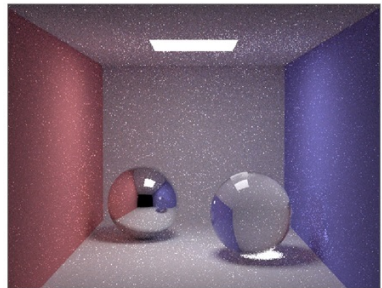
Institute of Computer Science II - Computer Graphics
University of Bonn

May 20, 2025





diffuse reflecting



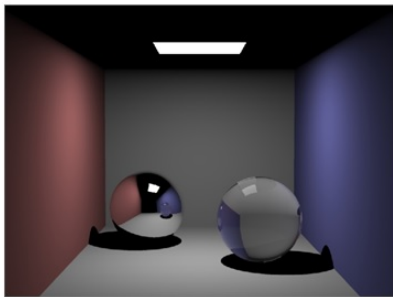
specular reflecting

Observations:

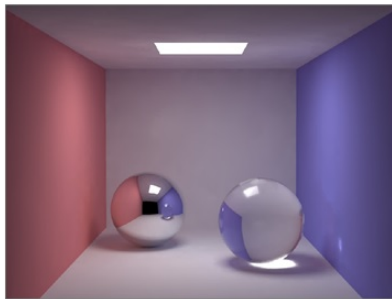
- ▶ Noise is perceived to be very different in both images
→ High-frequency and low-frequency details of multiple scattering
- ▶ Some scenes converge very slowly
→ Critical paths contain specular reflections/transmissions



Crucial Observation: The indirect illumination is **more important** than the direct one since it contributes most to a realistic look.



direct illumination



Direct+indirect illumination

Objective: Efficient calculation of the indirect illumination:

- ▶ Diffuse inter-reflections: “Bleeding” of colored objects on white walls, ...
- ▶ Caustics: Focused light like water waves or glass

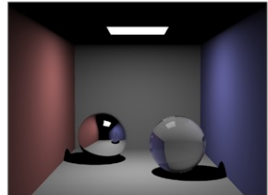


Observation:

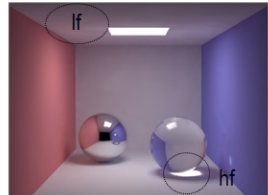
- ▶ Diffuse light transfer $L[D^*]DE$: Indirect illumination changes relatively slowly
→ low-frequent (lf)
- ▶ Caustics $LS[S^*]DE$: Strong “concentration” of incoming light
→ high-frequent (hf)

Idea: Cache and re-use indirect illumination
→ Photon-Map (Kd-Tree)

- ▶ Store incident indirect illumination
- ▶ Not directly coupled to geometry
- ▶ Independent of the viewpoint



Single scattering only



Single and multiple scattering



Photon Mapping:^{1, 2}

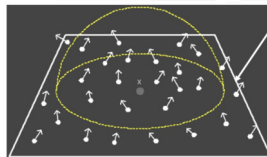
1. Photon Tracing:

- ▶ Emit photons from light sources
- ▶ Simulate light transport in the scene
- ▶ Store them in a Photon Map
- ▶ Construct Kd-Tree for Rendering



2. Rendering:

- ▶ Estimate irradiance using Photon Map
- ▶ Filter
- ▶ Split Rendering Equation into multiple parts and solve them
- ▶ Final Gathering

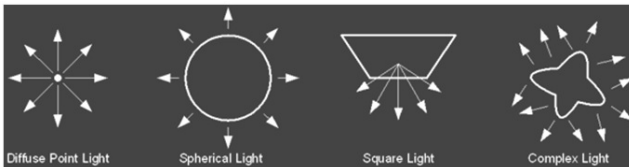


¹H. W. Jensen. "Global Illumination using Photon Maps". en. In: *Rendering Techniques '96*. Ed. by X. Pueyo and P. Schröder. Series Title: Eurographics. Vienna: Springer Vienna, 1996, pp. 21–30. URL: http://link.springer.com/10.1007/978-3-7091-7484-5_3 (visited on 06/12/2021).

²H. W. Jensen. *Realistic image synthesis using photon mapping*. AK Peters, Ltd., 2001.



Given: Light source with power Φ_{light}



If we sample n photons, each of them has the power

$$\Delta\Phi_{photon} = \frac{\Phi_{light}}{n}$$

- ▶ **Diffuse Point Light:** Rejection sampling
- ▶ **Spherical Light:** Generate uniformly sampled positions on the sphere using sampled directions on its hemisphere
- ▶ **Square light:** Same as Spherical Light
- ▶ **Complex Light:** Approximate emission function with spherical triangles and uniformly sample them



Example:

Algorithm Rejection Sampling of a Diffuse Point Light

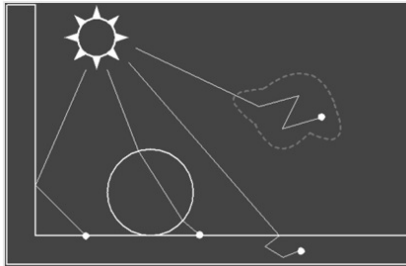
```
1:  $n = 0$ 
2: while  $n < n_{max}$  do
3:   while  $x^2 + y^2 + z^2 > 1$  do
4:      $x = \text{rand}(-1, 1)$ 
5:      $y = \text{rand}(-1, 1)$ 
6:      $z = \text{rand}(-1, 1)$ 
7:   end while
8:    $\mathbf{d} = (x, y, z)$ 
9:   Trace ray from light source in direction  $\mathbf{d}$ 
10:   $n = n + 1$ 
11:  Scale power of photon after tracing with  $1/n_{max}$ 
12: end while
```



Fact: Photons carry power of light

$$\Delta\Phi_{\text{photon}} [\text{W}]$$

through the scene, whereas rays carry radiance $L_{\text{ray}} \left[\frac{\text{W}}{\text{m}^2 \text{sr}} \right] !$



Scattering of photons is exactly the same as **Light Ray Tracing** for rays:

- ▶ Hit a specular surface: trace photon into reflection direction
- ▶ Hit diffuse surface: store photon and trace a new photon with a new direction chosen stochastically



Objective: To have a good efficiency, one wants the following properties:

- ▶ Do not trace unnecessary photons
- ▶ All photons should have approximately the same power to reduce variance

Idea: Use Russian Roulette of a uniformly sampled random variable $\xi \in [0, 1]$:

$\xi \in [0, d]$ $\rightarrow$ diffuse reflection

$\xi \in]d, d + s]$ $\rightarrow$ specular reflection

$\xi \in]d + s, 1]$ $\rightarrow$ absorption

So the photon power remains the same, but the number of photons decreases. This has the same effect as attenuating the power of all photons!



Problem: This only works if the radiant power is a 1-dimensional unit. However, light is typically represented as a 3-dimensional vector (e.g. RGB).

Idea: Extend Russian Roulette. Consider the probability of reflection as the maximum portion of energy that gets reflected:

$$\Pr(r) = \max \{d_r + s_r, d_g + s_g, d_b + s_b\} \in [0, 1],$$

where $\mathbf{f}_{BRDF,d} = (d_r, d_g, d_b)$ are the diffuse reflection coefficients of the RGB color bands of the BRDF and $\mathbf{f}_{BRDF,s} = (s_r, s_g, s_b)$ the respective specular reflection coefficients. Note that

$$\mathbf{f}_{BRDF} = \mathbf{f}_{BRDF,d} + \mathbf{f}_{BRDF,s}.$$

So $\Pr(r)$ is really the component with the highest reflection. Furthermore, the probabilities of diffuse and specular reflection are

$$\Pr(d) = \frac{d_r + d_g + d_b}{d_r + d_g + d_b + s_r + s_g + s_b} \Pr(r)$$
$$\Pr(s) = \frac{s_r + s_g + s_b}{d_r + d_g + d_b + s_r + s_g + s_b} \Pr(r)$$



Now, we can choose as sample $\xi \in [0, 1]$ and use Russian Roulette in the same way as before:

$$\begin{aligned}\xi \in [0, \text{Pr}(d)] & \rightarrow \text{diffuse reflection} \\ \xi \in]\text{Pr}(d), \text{Pr}(d) + \text{Pr}(s)] & \rightarrow \text{specular reflection} \\ \xi \in]\text{Pr}(d) + \text{Pr}(s), 1] & \rightarrow \text{absorption}\end{aligned}$$

However, we have to change the radiant power:

$$\Delta\Phi_{\text{photon scattered}} = \frac{f_{BRDF,k} \cdot \Delta\Phi_{\text{photon}}}{\text{Pr}(k)} \quad , \quad k \in \{d, s\}$$

So, if e.g. we have a specular reflection, we multiply the specular part of the BRDF with the radiant power of the photon and divide by the probability of this event. Similarly, if we have a diffuse reflection, we use the diffuse part and probability.



Finally, if a photon hits a surface with a diffuse component, it is stored:

```
struct photon {  
    float x,y,z;          // position  
    char p[4];            // power packed as 4 chars  
    char phi, theta;      // compressed incident direction  
    short flag;           // flag used in kdtree  
}
```

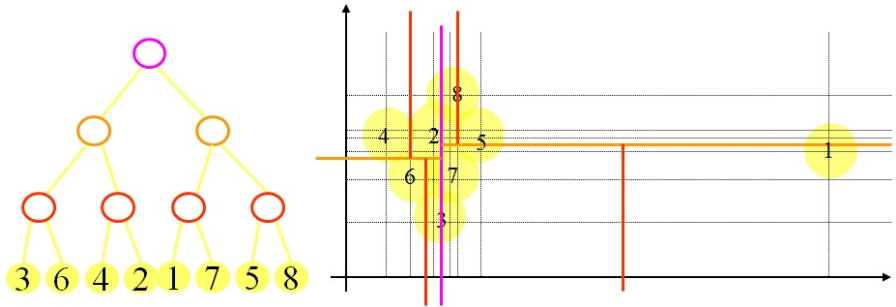
```
phi = 255 * (atan2(dy,dx)+PI) / (2*PI)  
theta = 255 * acos(dx) / PI
```

- ▶ The angles ϕ, θ are discretized so that 65536 different directions can be stored
- ▶ The radiant power is stored using 4 instead of 3 values to explicitly ensure memory alignment to 4 Bytes

Note: Photons are not stored on the geometry, they contain their positions themselves
→ construct a Kd-Tree



The Kd-Tree is constructed using all stored photons:



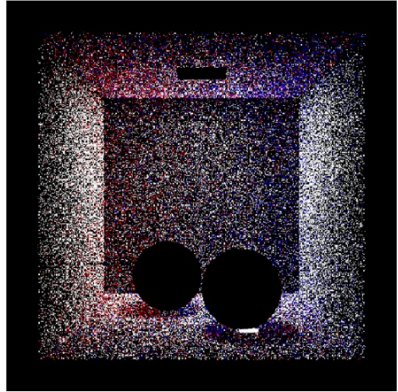
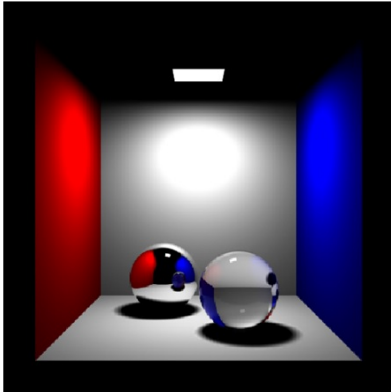
Since it is balanced, the time to find a particular photon is

$$\mathcal{O}(\log n)$$

→ Important for Nearest-Neighbor-Search



Example:



“Cornell Box” with glass and chrome sphere. **Left:** Result after ray tracing with direct illumination and specular reflection and transmission. **Right:** photons in photon map

To do rendering, we need to derive an estimator for the Rendering Equation

$$L_o(\mathbf{x}, \Omega_o) = L_e(\mathbf{x}, \Omega_o) + \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_i(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i$$

that uses the stored photons to compute the radiance. First, consider the definition of radiance: $L = \frac{d^2\Phi}{\cos(\theta) dA d\omega}$ which leads to

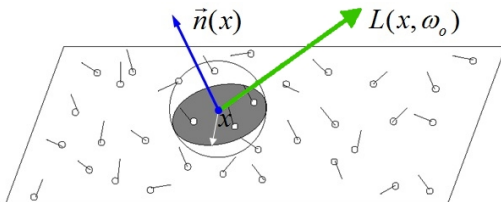
$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) + \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) \frac{d^2\Phi_i(\mathbf{x}, \Omega_i)}{\cos(\theta_i) dA d\omega_i} \cos(\theta_i) d\omega_i \\ &= L_e(\mathbf{x}, \Omega_o) + \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) \underbrace{\frac{d^2\Phi_i(\mathbf{x}, \Omega_i)}{dA}}_{=dE_i(\mathbf{x}, \Omega_i)} d\omega_i \end{aligned}$$

So we are integrating the irradiances weighted by the BRDF over the whole hemisphere. An estimator can be defined as:

$$L_o(\mathbf{x}, \Omega_o) \approx L_e(\mathbf{x}, \Omega_o) + \sum_{i=1}^n f_{BRDF}(\hat{\Omega}_i, \mathbf{x}, \Omega_o) \frac{\Delta\Phi_i(\mathbf{x}, \hat{\Omega}_i)}{\Delta A}.$$

An estimator can be defined as

$$L_o(x, \Omega_o) \approx L_e(x, \Omega_o) + \sum_{i=1}^n f_{BRDF}(\hat{\Omega}_i, x, \Omega_o) \frac{\Delta\Phi_i(x, \hat{\Omega}_i)}{\Delta A}$$



If we assume that the surface is locally flat around x , the projected area of the hemisphere is simply a circle with $\Delta A = \pi r^2$. So we get

$$L_o(x, \Omega_o) \approx L_e(x, \Omega_o) + \frac{1}{\pi r^2} \sum_{i=1}^n f_{BRDF}(\hat{\Omega}_i, x, \Omega_o) \Delta\Phi_i(x, \hat{\Omega}_i).$$



Problem: The irradiance is blurred out since every photon has equal weight.

Idea: Use a filter by weighting each photon individually according to its distance d_i to the query point x .

► **Cone-Filter:**

$$w_c(d_i) = 1 - \frac{d_i}{kr}$$

with radius r and user-specified constant k . Additionally, we need a normalization:

$$L_o(x, \Omega_o) \approx L_e(x, \Omega_o) + \frac{1}{\pi r^2} \sum_{i=1}^n \frac{w_c(d_i)}{1 - \frac{2}{3k}} f_{BRDF}(\hat{\Omega}_i, x, \Omega_o) \Delta\Phi_i(x, \hat{\Omega}_i)$$

► **Gauss-Filter:**

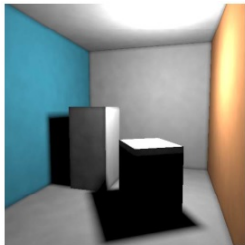
$$w_g(d_i) = \alpha \left(1 - \frac{1 - e^{-\beta \frac{d_i^2}{2r^2}}}{1 - e^{-\beta}} \right)$$

with constants $\alpha = 0.918, \beta = 1.953$ and radius r (more details in Pavicic (1990)³).

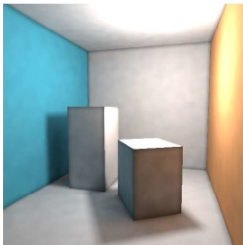
³M. J. Pavicic. "Convenient anti-aliasing filters that minimize "bumpy" sampling". In: *Graphics gems*. Academic Press Professional, Inc. 1990, pp. 144–146.



Example:



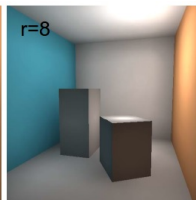
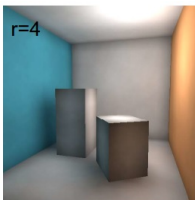
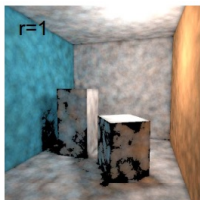
Only direct light
30000 Photons



1 diffuse reflection
40000 Photons



2 diffuse reflections
45000 Photons





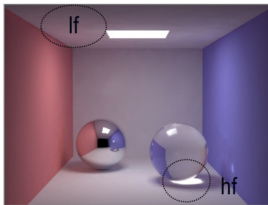
Now, we can use the Photon Map to solve the Rendering Equation. However, we split it into two Photon Maps:

Global Map

- ▶ **$L[D^*]DE$**
- ▶ Low density of photons
→ Indirect Illumination is low-frequency
- ▶ High energy per photon

Caustics Map

- ▶ **$LS[S^*]DE$**
- ▶ High density of photons
→ Caustics are high-frequency
- ▶ Low energy per photon



Benefits:

- ▶ Faster computation
- ▶ Higher accuracy
(→ variance of caustics)
- ▶ Less memory consumption



Since we now have two Photon Maps, we need to split the Rendering Equation to apply them to the respective parts. First, split the BRDF into a diffuse and specular component:

$$f_{BRDF} = f_{BRDF,d} + f_{BRDF,s}$$



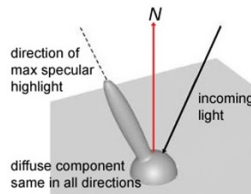
diffuse

+



specular

=



Then, split the incoming radiance into direct, caustic (specular) and diffuse part:

$$L_i = L_{i,l} + L_{i,c} + L_{i,d}$$



Using this, the Rendering Equation can be split into four integrals:

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) \\ &+ \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,l}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,s}(\Omega_i, \mathbf{x}, \Omega_o) (L_{i,c}(\mathbf{x}, \Omega_i) + L_{i,d}(\mathbf{x}, \Omega_i)) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,c}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,d}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \end{aligned}$$

Now, each part can be solved individually, some of them optionally in an approximate or an exact manner.



Direct lighting:

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) \\ &+ \int_{\mathcal{H}_i} \mathbf{f}_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,l}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} \mathbf{f}_{BRDF,s}(\Omega_i, \mathbf{x}, \Omega_o) (L_{i,c}(\mathbf{x}, \Omega_i) + L_{i,d}(\mathbf{x}, \Omega_i)) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} \mathbf{f}_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,c}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} \mathbf{f}_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,d}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \end{aligned}$$

Exact Solution: Ray Casting with light-source sampling

Approximation: Global Photon Map



Specular reflections (& refractions):

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) \\ &+ \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,l}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,s}(\Omega_i, \mathbf{x}, \Omega_o) (L_{i,c}(\mathbf{x}, \Omega_i) + L_{i,d}(\mathbf{x}, \Omega_i)) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,c}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,d}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \end{aligned}$$

Exact Solution: Path Tracing $\rightarrow$ Photon Map would require very high resolution



Caustics:

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) \\ &+ \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,l}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,s}(\Omega_i, \mathbf{x}, \Omega_o) (L_{i,c}(\mathbf{x}, \Omega_i) + L_{i,d}(\mathbf{x}, \Omega_i)) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,c}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,d}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \end{aligned}$$

Exact Solution: Path Tracing

Approximation: Caustics Photon Map



Indirect illumination:

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &= L_e(\mathbf{x}, \Omega_o) \\ &+ \int_{\mathcal{H}_i} f_{BRDF}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,l}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,s}(\Omega_i, \mathbf{x}, \Omega_o) (L_{i,c}(\mathbf{x}, \Omega_i) + L_{i,d}(\mathbf{x}, \Omega_i)) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,c}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \\ &+ \int_{\mathcal{H}_i} f_{BRDF,d}(\Omega_i, \mathbf{x}, \Omega_o) L_{i,d}(\mathbf{x}, \Omega_i) \cos(\theta_i) d\omega_i \end{aligned}$$

Exact Solution: Path Tracing (Option: Global Photon Map for Importance Sampling)

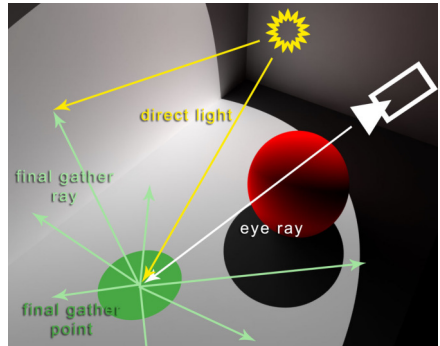
Approximation: Global Photon Map (Option: Final Gathering)



In the last part of the splitted Rendering Equation, we can additionally use Final Gathering. This technique reduces low-frequent artifacts of Photon Mapping.

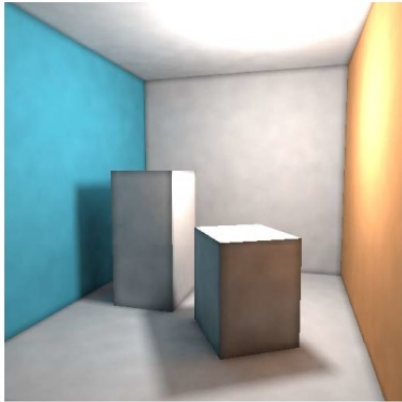
Idea: If an eye ray hits a diffuse surface, do not directly use the Global Photon Map. Do the following:

1. Shoot “Final Gather Rays” around the hemisphere like a Path tracer
2. Follow these rays and compute the radiance at their intersection points using the Global Photon Map (only if surface is diffuse there)
3. Summarize the collected radiances using a Monte-Carlo estimate

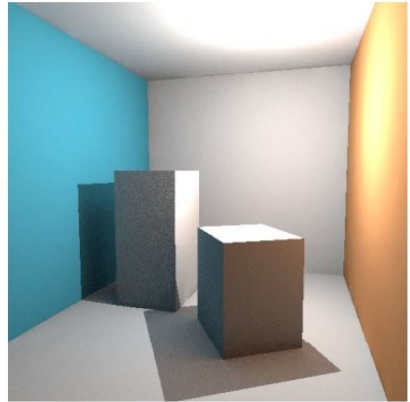




Example:



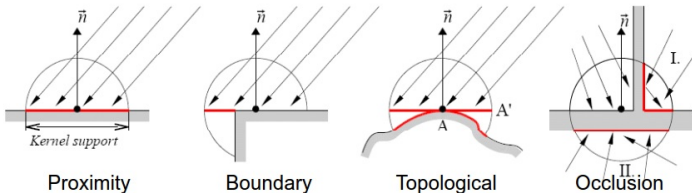
Direct visualization of Photon Map
several seconds



Final Gathering with 150 rays
several minutes



Unfortunately, Photon Mapping introduces a bias in our solution:



The main reason for this is the radiance estimate:

$$L_o(x, \Omega_o) \approx L_e(x, \Omega_o) + \frac{1}{\pi r^2} \sum_{i=1}^n f_{BRDF}(\hat{\Omega}_i, x, \Omega_o) \Delta\Phi_i(x, \hat{\Omega}_i)$$

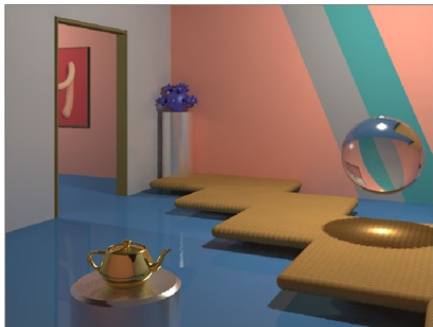
Since we use a small area $\Delta A = \pi r^2$, we indirectly apply a low-pass filter.

But: The solution of Photon Mapping converges to the correct solution if $r \rightarrow 0$. So a smaller radius leads to better results, but requires more photons.

→ Trade-off between accuracy and computation time

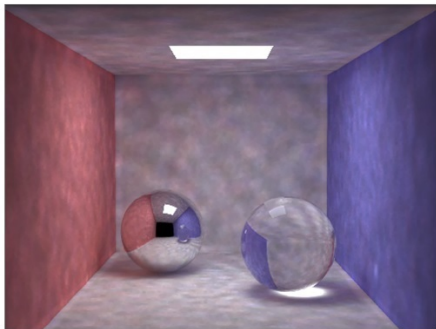


Results:





Results:



Global Photon Map with 100 Photons
to estimate irradiance (without Final Gathering)



Global Photon Map with 500 Photons
to estimate irradiance (without Final Gathering)



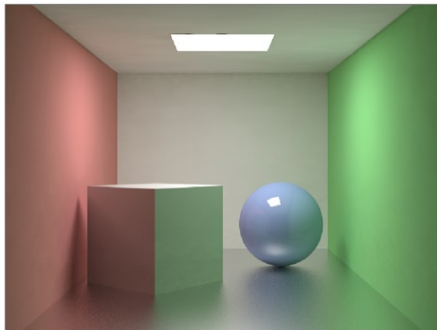
Results:



Total number of 500000 Photons, 100 Photons for estimation of irradiance, 20000 triangles for the water surface 11 Minutes computation time.



Results:





Biased and consistent image estimators

Definition

- An estimator F of an image I is called **unbiased** if

$$E(F - I) = 0 \Leftrightarrow E(F) = I.$$

If $E(F - I) = \beta \neq 0$ the estimator F is called **biased** with **bias** β .

For unbiased estimators the variance of F is measured as

$$Var(F) = E((F - I)^2)$$

It is called the estimator's **Mean Square Error** (MSE).

- An estimator $F(n)$ of I that is based on a sample sets of size n , is called **consistent** if for any $\epsilon > 0$

$$\lim_{n \rightarrow \infty} P[||F(n) - I|| > \epsilon] = 0.$$



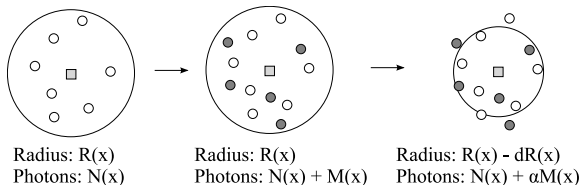
Observations:

- ▶ A path traced image estimate $F(1)$ with only one sample per pixel is an example of an unbiased estimator of the correct image.
- ▶ In contrast to a path traced image an image $F(n)$ generated by Photon Mapping results in a **biased** estimator of I , i.e. we don't expect to get the correct answer on average.
 - ▶ One main reason is that averaging a large number of blurry reconstructions of a region (e.g. in a caustic) will not result in a sharp reconstruction (of the caustic).
- ▶ Let $F(n)$ be image generated based on a Photon Map with of n photons. Combined with a suitable path tracer $F(n)$ is a consistent estimator of the correct image.
 - ▶ **Problem:** Unfortunately, because of the limited available memory we cannot increase the number of photons arbitrarily.

Summary: Standard Photon Mapping is a biased but consistent image estimator



Increasing the number of photons by combining photon maps⁴



- ▶ Calculating several independent photon maps can overcome the storage problem for an increasing number of photons.
- ▶ But simply averaging estimates from several runs does not have more detail than an individual photon map.
- ▶ After each photon tracing pass we find the new photons within the radius of each hit point, and we reduce the radius based on the newly added photons. This progressive radiance estimate ensures that the final value at each hit point will converge to the correct radiance value.

⁴T. Hachisuka, S. Ogaki, et al. "Progressive photon mapping". en. In: *ACM Transactions on Graphics* 27.5 (Dec. 2008), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1409060.1409083> (visited on 06/09/2021).



As before we use the photon density to estimate the radiance according to

$$L_o(x, \Omega_o) \approx L_e(x, \Omega_o) + \frac{1}{\pi R(x)^2} \sum_{i=1}^{N(x)} f_{BRDF}(\hat{\Omega}_i, x, \Omega_o) \Delta\Phi_i(x, \hat{\Omega}_i),$$

where $N(x)$ is the number of photons accumulated around a hit point x with radius $R(x)$. At x the density is estimated by

$$d(x) = \frac{N(x)}{\pi R(x)^2}.$$

If we perform one additional photon tracing step and find $M(x)$ photons within the radius $R(x)$ we get a new density

$$\hat{d}(x) = \frac{N(x) + M(x)}{\pi R(x)^2}$$

Vice versa, if we assume that the photon density is constant within a ball of radius $R(x)$ around x , we can compute number of photons $\hat{N}(x)$ within a any smaller disc around x . For a reduced radius $\hat{R}(x) = R(x) - dR(x)$ we get

$$\hat{N}(x) = \pi \hat{R}(x)^2 \hat{d}(x) = \pi (R(x) - dR(x))^2 \hat{d}(x)$$



Because $\hat{N}(x) > N(x)$ one can use a parameter $\alpha \in (0, 1)$ to control the fraction of photons added after every iteration:

$$\hat{N}(x) = N(x) + \alpha M(x).$$

Using this relation we can compute the actual reductions $dR(x)$ of the radius:

$$\pi(R(x) - dR(x))^2 \hat{d}(x) = \hat{N}(x)$$

$$\Leftrightarrow \pi(R(x) - dR(x))^2 \frac{N(x) + M(x)}{\pi R(x)^2} = N(x) + \alpha M(x)$$

$$\Leftrightarrow dR(x) = R(x) - R(x) \sqrt{\frac{N(x) + \alpha M(x)}{N(x) + M(x)}}$$

Finally, the reduced radius is computed as

$$\hat{R}(x) = R(x) - dR(x) = R(x) \sqrt{\frac{N(x) + \alpha M(x)}{N(x) + M(x)}}$$



Flux correction

- During rendering each hit point x stores the unnormalized total flux received premultiplied by the BRDF. For $N(x)$ photons this quantity is estimated as

$$\tau_{N(x)}(\mathbf{x}, \boldsymbol{\Omega}_o) = \sum_{i=1}^{N(x)} f_{BRDF}(\boldsymbol{\Omega}_i, \mathbf{x}, \boldsymbol{\Omega}_o) \Phi'_i(\mathbf{x}, \boldsymbol{\Omega}_i),$$

where $\boldsymbol{\Omega}_o$ is the outgoing direction at the hit point $\mathbf{x}$, $\boldsymbol{\Omega}_i$ is the incoming direction of the incident photon i and $\Phi'_i(\mathbf{x}, \boldsymbol{\Omega}_i)$ is the unnormalized flux carried by the photon i .

- Note, that the the flux at this stage is not divided by the number of emitted photons as in standard photon mapping.
- If the radius was constant we could simply add $\tau_{M(x)}(\mathbf{x}, \boldsymbol{\Omega}_o)$ computed in a second run using the $M(x)$ photons in the neighbourhood of $\mathbf{x}$ to the current estimate $\tau_{N(x)}(\mathbf{x}, \boldsymbol{\Omega}_o)$, but since the radius is reduced we need to account for the photons that fall outside the reduced radius.



Flux correction cont.

- To account for the reduced radius we assume that the illumination and the photon density within the disc is constant, which results in the following adjustment:

$$\begin{aligned}\tau_{\hat{N}(x)}(\mathbf{x}, \boldsymbol{\Omega}_o) &= (\tau_{N(x)}(\mathbf{x}, \boldsymbol{\Omega}_o) + \tau_{M(x)}(\mathbf{x}, \boldsymbol{\Omega}_o)) \frac{\pi \hat{R}(x)^2}{\pi R(x)^2} \\ &= \tau_{N(x)+M(x)}(\mathbf{x}, \boldsymbol{\Omega}_o) \frac{\pi \left(R(x) \sqrt{\frac{N(x) + \alpha M(x)}{N(x) + M(x)}} \right)^2}{\pi R(x)^2} \\ &= \tau_{N(x)+M(x)}(\mathbf{x}, \boldsymbol{\Omega}_o) \frac{N(x) + \alpha M(x)}{N(x) + M(x)}.\end{aligned}$$



Radiance evaluation

At the i -th pass during the progressive computation, the local indirect radiance the following quantities are updated and stored at the hit points:

- ▶ Position $\mathbf{x}$ of the hit point,
- ▶ search radius $R_i(\mathbf{x})$,
- ▶ local accumulated photon count within radius $N_i(\mathbf{x})$,
- ▶ accumulated (unnormalized) flux times BRDF τ_i .

Based on this quantities the radiance is evaluated as follows:

$$\begin{aligned} L_o(\mathbf{x}, \Omega_o) &\approx L_e(\mathbf{x}, \Omega_o) + \frac{1}{\pi \hat{R}(\mathbf{x})^2} \sum_{i=1}^{N_i(\mathbf{x})} f_{BRDF}(\hat{\Omega}_i, \mathbf{x}, \Omega_o) \Delta\Phi_i(\mathbf{x}, \hat{\Omega}_i) \\ &= L_e(\mathbf{x}, \Omega_o) + \frac{1}{\pi R_i(\mathbf{x})^2} \frac{\tau_i(\mathbf{x})}{N_e(i)}, \end{aligned}$$

where $N_e(i)$ is the total number of emitted photons after i passes.



Increasing the number of photons using Progressive Photon Mapping⁵:

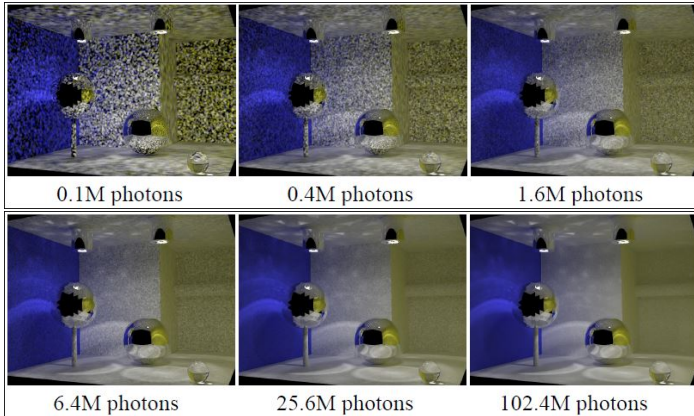


Figure: Adding more photons makes the image sharper and reduces low frequency noise.

⁵T. Hachisuka, S. Ogaki, et al. "Progressive photon mapping". en. In: *ACM Transactions on Graphics* 27.5 (Dec. 2008), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1409060.1409083> (visited on 06/09/2021).



Results:

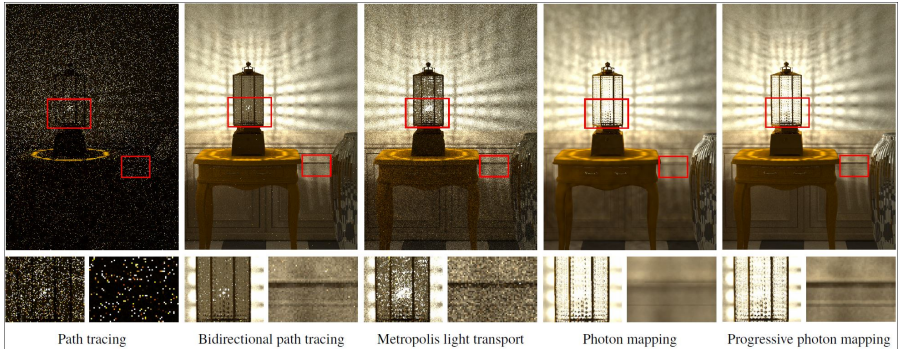


Figure: A glass lamp illuminates a wall and generates a complex caustics lighting pattern on the wall. The lighting seen through the lamp is particularly difficult for other methods. Photon mapping is significantly better at capturing the caustics lighting seen through the lamp, but the final quality is limited by the memory available for the photon map and it lacks the fine detail in the illumination. Progressive photon mapping provides an image with substantially less noise in the same render time as the Monte Carlo ray tracing methods and the final quality is not limited by the available memory.



Increasing the number of photons using Progressive Photon Mapping⁶:

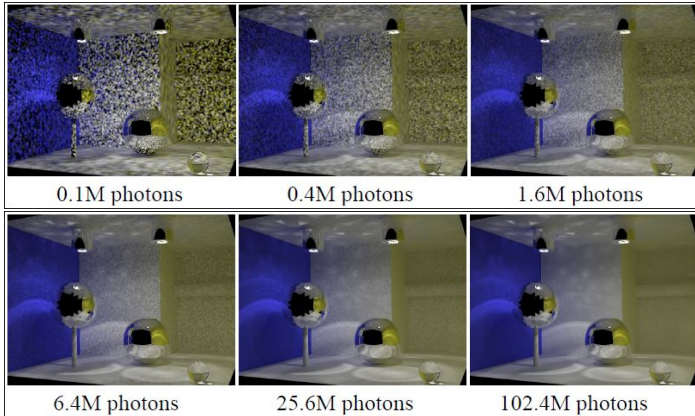


Figure: Adding more photons makes the image sharper and reduces low frequency noise.

⁶T. Hachisuka, S. Ogaki, et al. "Progressive photon mapping". en. In: *ACM Transactions on Graphics* 27.5 (Dec. 2008), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1409060.1409083> (visited on 06/09/2021).



Stochastic Progressive Photon Mapping⁷

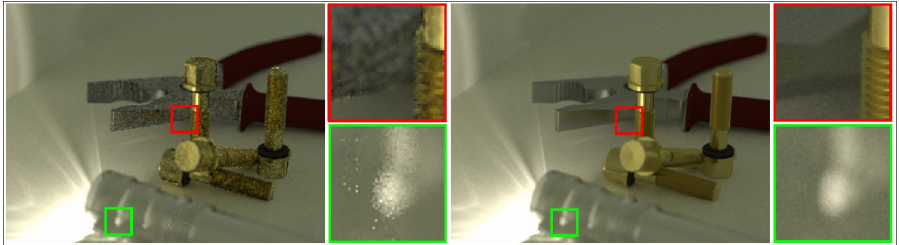


Figure: Tools with a flashlight. The scene is illuminated by caustics from the flashlight, which cause SDS paths on the flashlight and highly glossy reflections of caustics on the bolts and plier. The flashlight and the plier are out of focus. Using the same rendering time, our method (right) robustly renders the combination of the complex illumination setting and the distributed ray tracing effects where progressive photon mapping is inefficient (left).

⁷T. Hachisuka and H. W. Jensen. “Stochastic progressive photon mapping”. en. In: *ACM Transactions on Graphics* 28.5 (Dec. 2009), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1618452.1618487> (visited on 06/10/2021).



Basic idea

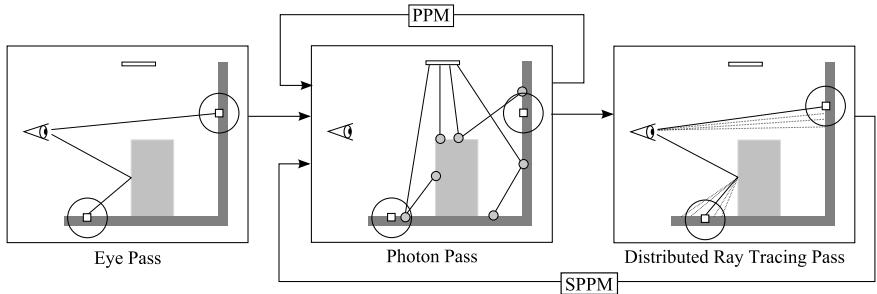


Figure: Difference between the algorithms of progressive photon mapping (PPM) and stochastic progressive photon mapping (SPPM). In order to compute the average radiance values, SPPM adds a new distributed ray tracing pass after each photon tracing pass. The photon tracing algorithm itself stays the same, but PPM uses a fixed set of hit points (shown as squares), whereas SPPM uses randomly generated hit points by the distributed ray tracing pass.

Update of statistics If $M_i(x)$ photons are found within the search radius $R_i(x)$ during pass i , the progressive radiance estimate update the statistics as:

PPM:

$$\begin{aligned} N_{i+1}(x) &= N_i(x) + \alpha M_i(x) \\ R_{i+1}(x) &= R_i(x) \sqrt{\frac{N_i(x) + \alpha M_i(x)}{N_i(x) + M_i(x)}} \\ \Phi_i(x, \Omega_o) &= \sum_{j=1}^{M_i(x)} f_r(\Omega, x, \Omega_o) \Phi_j(x_j, \Omega_j) \\ \tau_{i+1}(x, \Omega_o) &= \left(\tau_i(x, \Omega_o) + \Phi(x, \Omega_o) \right) \frac{R_{i+1}(x)^2}{R_i(x)^2}, \end{aligned}$$

SPPM:

$$\begin{aligned} N_{i+1}(S) &= N_i(S) + \alpha M_i(x_i) \\ R_{i+1}(S) &= R_i(S) \sqrt{\frac{N_i(S) + \alpha M_i(x_i)}{N_i(S) + M_i(x_i)}} \\ \Phi_i(x_i, \Omega_o) &= \sum_{j=1}^{M_i(x_i)} f_r(\Omega, x_i, \Omega_o) \Phi_j(x_j, \Omega_j) \\ \tau_{i+1}(S, \Omega_o) &= \left(\tau_i(S, \Omega_o) + \Phi(x_i, \Omega_o) \right) \frac{R_{i+1}(S)^2}{R_i(S)^2}, \end{aligned}$$

where $\alpha \in (0, 1)$ is a user-defined parameter, $f_r = f_{BRDF}$, $\Phi_j(x, \Omega_j)$ is the flux of photon j , and Ω_j is the incoming direction of photon j . where x_i is a randomly generated position within S in pass i and $N_i(S)$ is the shared local photon count.

Note that the updating procedure is the same as before, except that our formulation uses a randomly generated position x_i in S .

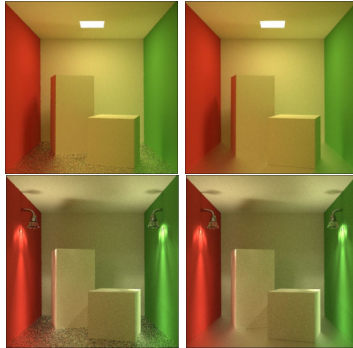


Figure: Left PPM, Right SPPM. Cornell box with a glossy floor. The BRDF of the floor is the modified Phong model with a glossiness value of 25. The results of PPM are noisy because glossy reflections cause large variations of photon contributions to viewing directions. The bottom row replaced the area light source in the original Cornell box with two wall lights. Even though this change is simple from a user's perspective, it makes unbiased methods inefficient because of highly glossy reflections of caustics from the wall lights. SPPM is still as robust as PPM for this illumination setting, while rendering glossy reflections with less noise in the same rendering time.

In **Memoryless PPM**⁸ a pixel value c is computed as follows:

$$c = \int W(x, \omega) L(x, \omega) dx d\omega,$$

where we integrate over all scene surfaces and the hemisphere of directions at each point, $L(x, \omega)$ is the reflected radiance and $W(x, \omega)$ is a weighting function that describes the contribution of $L(x, \omega)$ for each surface location x and direction ω to the pixel value.

- $W(x, \omega)$ can include a pixel filter for antialiasing, or effects such as motion blur, depth of field, and glossy reflections.
- In general, it is evaluated by tracing paths from the eye.

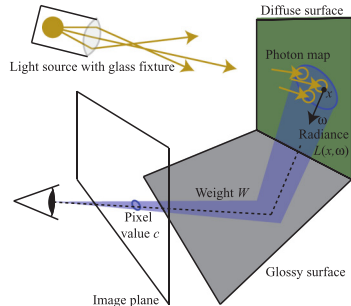


Figure: typical scenario for progressive photon mapping. A light fixture enclosed by glass illuminates a scene that contains diffuse and glossy surfaces.

⁸C. Knaus and M. Zwicker. "Progressive photon mapping: A probabilistic approach". en. In: *ACM Transactions on Graphics* 30.3 (May 2011), pp. 1–13. URL: <https://dl.acm.org/doi/10.1145/1966394.1966404> (visited on 06/10/2021).



In Memoryless PPM the above integral, i.e. a complete pixel measurement estimate, is obtained by a Monte Carlo estimate and the reflected radiance at $\mathbf{x}$ in direction of $\boldsymbol{\omega}$ is computed using a photon map:

$$\bar{c}_N = \frac{1}{N} \sum_1^N \frac{1}{p_e(\mathbf{x}_i, \boldsymbol{\omega}_i)} W(\mathbf{x}_i, \boldsymbol{\omega}_i) (L(\mathbf{x}_i, \boldsymbol{\omega}_i) + \epsilon_i),$$

where $\bar{c}_N$ denotes the estimate for a pixel value using N samples, that is, eye paths, $(\mathbf{x}_i, \boldsymbol{\omega}_i)$ denote position and direction of surface intersections of paths generated by ray tracing starting from the eye, $p_e(\mathbf{x}_i, \boldsymbol{\omega}_i)$ their probability density and ϵ_i is the radiance estimation error of the i -th sample.

A crucial property of progressive photon mapping is that each sample is evaluated using a new photon distribution, that is, a new photon map. Therefore, ϵ_i is interpreted as a sample of a random variable, since the error of the radiance estimate depends on the random distribution of photons.

The average error of the photon map radiance estimate over N samples is given by

$$\bar{\epsilon}_N = \frac{1}{N} \sum_{i=1}^N \epsilon_i.$$

It can be shown that if the variance and the expected value of the average error $\bar{\epsilon}_N$ both approach zero as $N \rightarrow \infty$ then the estimator $\bar{\epsilon}_N$ is consistent:

$$Var[\bar{\epsilon}_N] \rightarrow 0 \implies Var[\bar{c}_N] \rightarrow 0 \quad (1)$$

$$E[\bar{\epsilon}_N] \rightarrow 0 \implies E[\bar{c}_N] \rightarrow c. \quad (2)$$

As a consequence, with each new sample the variance of the error ϵ_N at every iteration is allowed to increase by a factor

$$\frac{Var[\epsilon_{N+1}]}{Var[\epsilon_N]} = \frac{N+1}{N+\alpha}$$

for some constant $\alpha \in (0, 1)$.

The parameter α controls how quickly the variance is allowed to increase in each iteration. It effectively determines a trade-off between vanishing variance and expected value of the average error.

Relating variance and kernel radius

In PPM an estimate of $L(\mathbf{x}, \omega)$ is computed by

$$L(\mathbf{x}, \omega) \approx \frac{1}{M} \sum_{j=1}^M k_r(x_j - x) \gamma_j,$$

where the sum is over all M photons in the current photon map, x_j is the position of the j -th photon, k_r is a kernel function that determines the weight of each photon in the local average and γ_j is the contribution of each photon including the BRDF and the photon value which represents the contribution of the photon path divided by the probability with which the path was sampled.

Variance of the expected value of the error

Let $\epsilon(\mathbf{x}, r)$ be the difference between the radiance estimate at position $\mathbf{x}$ and using a kernel k_r of scale r , and the true radiance $L(\mathbf{x}, \omega)$:

$$\epsilon(\mathbf{x}, r) = \frac{1}{N} \sum_{j=1}^M k_r(x_j - \mathbf{x}) \gamma_j - L(\mathbf{x}, \omega).$$

Then its variance can be approximated as

$$\text{Var}[\epsilon(\mathbf{x}, r)] \approx \frac{(\text{Var}[\gamma] + E[\gamma]^2) p_l}{M r^2} \int_{\mathbb{R}^2} k(\mathbf{u}) d\mathbf{u},$$

where p_l is the (constant) density of photons in the support of the kernel k_r .

This confirms the intuition that variance decreases linearly with the number of emitted photons M and the square of the kernel scale.

Specifying a variance $\text{Var}[\epsilon(\mathbf{x}, r)]$, one obtains the corresponding kernel radius:

$$r(\mathbf{x}, \text{Var}[\epsilon]) \approx \frac{(\text{Var}[\gamma] + E[\gamma]^2) p_l}{M \text{Var}[\epsilon]} \int_{\mathbb{R}^2} k(\mathbf{u}) d\mathbf{u},$$



Expected Value of Average Error

Using this relation between variance and radius we get

$$\frac{r_{i+1}^2}{r_i^2} \frac{Var[\epsilon_{N+1}]}{Var[\epsilon_N]} = \frac{N+1}{N+\alpha}$$

Assuming an initial radius r_1 from this we get an explicit equation for radius r_i :

$$r_i^2 = r_1^2 \left(\prod_{k=1}^{i-1} \frac{k+\alpha}{k} \right) \frac{1}{i}.$$

Since the expected error is proportional to the square radius, the expected error of the i -th sample is

$$E[\epsilon_i] = E[\epsilon_1] \left(\prod_{k=1}^{i-1} \frac{k+\alpha}{k} \right) \frac{1}{i}.$$

and the expected value of the average error is

$$E[\bar{\epsilon}_i] = E[\epsilon_1] \left(1 + \sum_{i=2}^N \left(\prod_{k=1}^{i-1} \frac{k+\alpha}{k} \right) \frac{1}{i} \right).$$

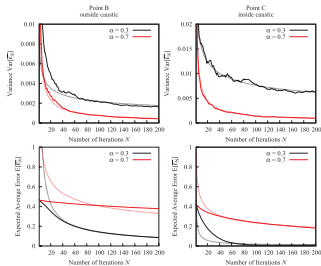


Figure: Vanishing behavior of variance and expected error for pixels B and C. The plots in the top row show the variance $\text{Var}[\tilde{N}]$ and the plots in the bottom row show the expected average error $E[\tilde{N}]$ as a function of the number of iterations N . We plot the empirically estimated variance and expected average error (solid lines) and the asymptotic behavior of the variance $O\left(\frac{1}{N^{1-\alpha}}\right)$ and expected average error $O\left(\frac{1}{N^{1-\alpha}}\right)$ (dotted lines). We show results for $\alpha = 0.3$ and $\alpha = 0.7$, using 100,000 photons in each iteration step. The reference radius is determined as the kNN distance in the photon maps with $k = 100$.

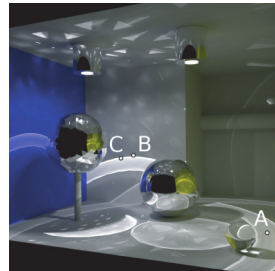


Figure: This figure shows three characteristic points sampled for the statistics on the left hand side. Point A lies in a homogenous region, points B and C lie just outside and just inside the caustic.



- [1] T. Hachisuka and H. W. Jensen. “Stochastic progressive photon mapping”. en. In: *ACM Transactions on Graphics* 28.5 (Dec. 2009), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1618452.1618487> (visited on 06/10/2021).
- [2] T. Hachisuka, S. Ogaki, and H. W. Jensen. “Progressive photon mapping”. en. In: *ACM Transactions on Graphics* 27.5 (Dec. 2008), pp. 1–8. URL: <https://dl.acm.org/doi/10.1145/1409060.1409083> (visited on 06/09/2021).
- [3] H. W. Jensen. “Global Illumination using Photon Maps”. en. In: *Rendering Techniques '96*. Ed. by X. Pueyo and P. Schröder. Series Title: Eurographics. Vienna: Springer Vienna, 1996, pp. 21–30. URL: http://link.springer.com/10.1007/978-3-7091-7484-5_3 (visited on 06/12/2021).
- [4] H. W. Jensen. *Realistic image synthesis using photon mapping*. AK Peters, Ltd., 2001.
- [5] A. S. Kaplanyan and C. Dachsbacher. “Adaptive progressive photon mapping”. en. In: *ACM Transactions on Graphics* 32.2 (Apr. 2013), pp. 1–13. URL: <https://dl.acm.org/doi/10.1145/2451236.2451242> (visited on 06/10/2021).



- [6] C. Knaus and M. Zwicker. “Progressive photon mapping: A probabilistic approach”. en. In: *ACM Transactions on Graphics* 30.3 (May 2011), pp. 1–13. URL: <https://dl.acm.org/doi/10.1145/1966394.1966404> (visited on 06/10/2021).
- [7] Z. Lin, S. Li, X. Zeng, C. Zhang, J. Jia, G. Wang, and D. Manocha. “CPPM: chi-squared progressive photon mapping”. en. In: *ACM Transactions on Graphics* 39.6 (Nov. 2020), pp. 1–12. URL: <https://dl.acm.org/doi/10.1145/3414685.3417822> (visited on 06/09/2021).
- [8] M. J. Pavicic. “Convenient anti-aliasing filters that minimize “bumpy” sampling”. In: *Graphics gems*. Academic Press Professional, Inc. 1990, pp. 144–146.
- [9] S. Zhu, Z. Xu, H. W. Jensen, H. Su, and R. Ramamoorthi. “Deep Kernel Density Estimation for Photon Mapping”. en. In: *Computer Graphics Forum* 39.4 (July 2020), pp. 35–45. URL: <https://onlinelibrary.wiley.com/doi/10.1111/cgf.14052> (visited on 06/09/2021).